ChatGPT

复现并嵌入 SAPIEN 的工程实施报告

执行摘要

这篇工作的**已发布代码**可以复现出一条“真实采集 → 3DGS 背景 → 物体网格/关节推断 → 仿真合成数据”的主干思路，但**不能直接原样嵌入你现有的 SAPIEN 项目**。原因非常明确：官方仓库 `Maxwell-Zhao/RoboSimGS` 当前把数据生成部分实现为 **Genesis + LeRobot + 自定义 Gaussian 渲染器**，而不是 SAPIEN；并且 README 明确写了 **Camera Pose Alignment 代码尚未发布**。因此，最现实、风险最低的路线，不是“照搬论文代码到 SAPIEN”，而是**保留其资产生成链路**，然后**重写仿真层与渲染桥接层**到 SAPIEN。[1]

对你的项目，我的直接建议是：**把复现任务拆成两层**。第一层是“资产层”，包括真实场景背景 3DGS、真实物体 mesh、分部件分割、URDF 生成、物理参数估计和坐标对齐；这一层可最大化沿用 `RoboSimGS`、`COLMAP`、`Grounded-SAM-2`、`Open3D`、`trimesh` 等仓库。第二层是“仿真层”，即把输出的 `mesh / urdf / transform / camera / physics.json` 接入 SAPIEN，并用 SAPIEN 负责动力学、碰撞、机器人控制、相机与数据导出；3DGS 背景则不建议塞进 SAPIEN 内核里硬渲染，而是通过 `gsplat` 或兼容渲染器做**相机同步的外部合成**。这条路线与 `SplatSim` “把 Gaussian Splatting 作为现有模拟器之上的渲染层”的思路一致，但更适合你的 SAPIEN 工程。[2]

从工程可执行性上看，这篇论文的**最关键复现要点**不是“训练策略网络”，而是四个可独立落地的模块：**现实物体建模、物理属性与纹理模拟、自动生成数据、以及 3DGS 背景与 mesh 的混合渲染**。其中，现实物体建模和世界对齐已有相对明确的仓库线索；物理属性估计已经有发布代码，但其实际实现是“BLIP2/CLIP 粗分类 + OpenAI [3] API 细化 JSON 参数”的启发式 MLLM 管线，而不是完全自洽的物理识别器；纹理/PBR 可通过 `MaterialPalette` 或可选的 Tencent [4] `Hunyuan3D-2.1` 补上；背景 3DGS 建议优先使用 Apache-2.0 的 `gsplat`，而不是研究用途限制更强的官方 `gaussian-splatting`。[5]

对问题的直接回答

问题 1：如何把现实物体映射到仿真环境？

可以，但要接受一个现实：**官方已发布实现仍然是“半自动 + 人在环”**。`RoboSimGS` README 明确写出：背景采用 3DGS，并使用 `Nerfstudio` 的 Feature Splatting 做机器人与背景分离；物体重建则直接依赖手机扫描得到高质量 mesh；世界坐标对齐通过真实机器人点云与仿真机器人点云做配准，且因为 ICP 不解尺度，官方流程要求用户交互式调节 scale 后再做 ICP。仓库中的 `icp.py` 也证实了这一点：脚本接收真实点云、仿真点云和一个手动给定的 `size` 缩放值，再做最近邻 ICP 并输出 4×4 变换矩阵。也就是说，**现实物体映射到仿真**的最稳妥路径是：`扫描/重建 → 部件分割 → URDF 生成 → 真实/仿真几何对齐 → 相机对齐`，其中目前真正全自动的只有一部分。[6]

在“重建表示”上，我建议你把**物体与背景分开建模**。物体优先走 **mesh 路线**，因为 SAPIEN/URDF/碰撞/质量参数都天然围绕 mesh 与 link 组织；背景优先走 **3DGS 路线**，因为它的目标是高保真视觉，不承担复杂动力学。论文官方仓库当前对物体的描述就是“手机扫描得到高质量 mesh”，而背景才是 3DGS。若你想全开源替代手机扫描，可用 `COLMAP` 完成 SfM/MVS 基础重建，再用 `Open3D` / `trimesh` 清洗网格；若只需要快速得到带纹理资产，`Hunyuan3D-2.1` 可以作为补充，但它更适合“资产增强”，不适合代替精确的真实物体几何真值。[7]

在“关节/关节参数推断”上，已发布的 `articulation_inference.py` 说明官方实现目前仍依赖 **交互式分割 + 启发式命名 + GPT 参数推荐**。它支持 `--interactive`、`--skip-segmentation`、`--lid`、`--body`、`--use-

`sam-clip`、`--use-gpt-params`等模式；分割后用关键词启发式把部件识别为`lid/body`，然后进入`URDFGenerationPipeline`生成URDF，关节上下限、effort、velocity、friction、damping则可由GPT推荐或手工覆盖。这意味着，如果你要在SAPIEN项目里做成“可复现的工程版”，**不应把它包装成“Fully automatic articulation inference”**，而应当设计成“交互优先、自动建议、人工复核”的流水线。[8]

在“标注与自动化工具链”上，最实用的方案是：**视频采集 + COLMAP相机 + Grounded-SAM-2 / SAM 2分割 + Open3D/trimesh几何处理 + URDF组装 + Open3D/CloudCompare/ICP对齐 + SAPIEN导入**。如果你需要link级标签导出，还要预先在导入SAPIEN时维护一份`link_name -> 自定义实例ID`映射。因为在SAPIEN 3里，已有公开issue指出`camera.get_picture("Segmentation")`返回`(visual_id, entity_id)`，但用户找不到稳定的`entity_id -> link_name`官方API，旧接口也已移除。对你的工程，这意味着**不要把link标注绑定到SAPIEN的内部ID**，而要在你的资产加载器上自己维护一份稳定映射表。[9]

问题2：论文中仿真环境的具体实现细节是什么，与SAPIEN是否兼容？

就**已发布代码**而言，论文的仿真环境核心不是SAPIEN，而是**Genesis**。`RoboSimGS` README直接写明代码建立在`Genesis`和`Lerobot`之上；`DataGeneration/base_task.py`则清楚地`import genesis as gs`，在`DataCollector`初始化里调用`gs.init(backend=gs.gpu)`。当`use_gs=True`时，它会加载自定义`Gaussians.render.Renderer()`并读取`exports/.../splat-transform.ply`；而实际任务场景通过`gs.Scene(...)`创建，设置`substeps=60`，使用`gs.renderers.Rasterizer()`，并向场景里加入`gs.materials.Rigid()`的mesh/plane/box/camera/light。换言之，**官方公开实现是“Genesis动力学 + 自定义Gaussian背景渲染 + 数据增强导出”**。[10]

更细看实现细节，当前公开demo更偏向**刚体抓取场景**：它把任务目标物实例化为`gs.materials.Rigid()`，mesh以`gs.morphs.Mesh(..., collision=True, visualization=True)`加入场景，地面是plane，目标区域是box；控制上使用Genesis的`inverse_kinematics`、`control_dofs_position`；视觉上做了两类简单增强：**相机位置/看向抖动**和**灯光颜色/强度随机化**；数据则导出`actions.npy`、`qpos.npy`与逐帧图片/视频。这些细节对你很重要，因为它说明论文的已放出代码，并不是一个完整的“统一资产→统一软体/刚体→统一视觉”的大系统，而是一套**已经够用但仍在快速演化中的demo级主线实现**。[11]

与SAPIEN的兼容性方面，我的结论是：**高兼容，但不是二进制兼容；需要API级适配层**。SAPIEN原生支持articulation tree、`revolute/prismatic/fixed`关节、`set_joint_properties`、URDF/MJCF导入、`create_physical_material`、默认物理材质、以及rasterization / ray tracing两套渲染路径；ManiSkill文档也说明，它支持导入URDF/MJCF，但控制器、挂载传感器、机器人类定义要你自己补齐。这意味着，**论文的资产产物（mesh、urdf、相机、变换、物理参数）可以接到SAPIEN；但论文的Genesis仿真脚本本身要重写**。[12]

你在SAPIEN中最需要关注的四个适配点是：**场景API、关节模型、动力学参数、渲染管线**。场景API上，Genesis用`scene.add_entity(...)`和material/morph/surface三段式构造；SAPIEN则是`create_actor_builder()` / `create_urdf_loader()` / `ArticulationBuilder`。关节模型上，Genesis demo直接用IK与DOF控制；SAPIEN则要通过joint drive / controller体系组织。动力学参数上，Genesis的软体/MPM/PBD能力更广，而SAPIEN强项是刚体与articulated-body；因此如果你的目标是论文里的zero-shot manipulation数据集生成，**第一版应先把可操作物体限制为刚体或准刚体**。渲染上，SAPIEN非常适合渲染robot/object/seg/depth，但**不适合直接作为3DGS的宿主渲染器**；背景3DGS应单独渲染后合成。[13]

问题3：如何生成物理属性与背景？

物理属性方面，`RoboSimGS/Articulation/physics_estimation.py`已经给出了一条相当具体的MLLM管线：它先读取图像，调用BLIP2生成caption，再让`predict_candidate_materials`给出粗材料和粗密度候选；接着用CLIP在候选材料中做相似度选择；随后依据“刚性/非刚性”分支，去查`material_table.json`或`material_table_pbd.json`的粗参数，并让API产出一个**可被代码直接parse的JSON**。刚性侧JSON包含

`Density (kg/m³)`、`Young's modulus (MPa)`、`Poisson's ratio`；PBD 侧包含 `Density (kg/m³)`、`static_friction`、`kinetic_friction`、`stretch_compliance`。这说明论文已发布代码的“物理属性估计”本质上是**视觉-语言先验 + 材料表查表 + LLM 数值细化**。[14]

把这套思路接到 SAPIEN 时，我建议你把输出拆成两层：**材质层**与**动力学层**。材质层用于渲染，保存为 `pbr.json + textures/`；动力学层用于碰撞和质量，保存为 `physics.json`。前者记录 base color / metallic / roughness / normal / transmission；后者记录 `density / static_friction / dynamic_friction / restitution / youngs_modulus / poisson / mass_scale / inertia_mode`。其中，质量分配不要直接让 LLM 硬猜，而应当用 `density × watertight mesh volume` 得到总质量，再用 `trimesh` 或你自己的体素/四面体离散做 link 级质量和惯量分配。这样做既保留了论文的 MLLM 思路，也把它落成一个可以在 SAPIEN/URDF/自定义加载器里稳定消费的数据接口。[15]

纹理与 PBR 材质方面，你有两条可落地路线。第一条是更稳、更适合真实物体的：用 `MaterialPalette` 从真实图像/区域 mask 中抽取材料与纹理概念，然后烘焙成 PBR 图；第二条是更方便但许可证更敏感的：使用 `Hunyuan3D-2.1` 的 shape/paint 管线直接生成“production-ready PBR material”。它的 README 明确给出了形状与贴图双 pipeline、VRAM 需求以及示例 API，但其许可证不是 MIT/Apache，而是 Tencent 社区许可，并带有地域与用途限制。对工程项目而言，我建议 **默认采用 MIT 的 MaterialPalette 做真实图像材质抽取**，把 Hunyuan3D 作为可选增强，而不是主依赖。[16]

背景方面，最实用的体系是：**3DGS 只负责背景，robot/object 仍由 SAPIEN 渲染**。这既符合 `RoboSimGS` 的公开思路，也与 `SplatSim` “Gaussian Splatting 作为现有模拟器上方渲染器”的路线一致。实现上，推荐使用 `gsplat` 而不是官方 `graphdeco-inria/gaussian-splatting` 作为你项目内嵌的渲染后端：一方面，`gsplat` 是 Apache-2.0，且官方说明其更快、更省显存；另一方面，`graphdeco` 官方实现的许可证明确是研究用途导向、限制更强。对于你的 SAPIEN 项目，这个选择会直接影响二次分发和后续产品化风险。[17]

推荐仓库与选型

下表只列**我最终推荐**、且与“复现论文并嵌入 SAPIEN”直接相关的仓库。仓库名可直接作为检索入口；“来源”列对应的是 GitHub 仓库主页或仓库内关键文件页面。

功能分类	推荐仓库	关键文件 / 脚本	License / 活跃度	选型意见	来源
论文官方主线	`Maxwell-Zhao/RoboSimGS`	`README.md`、`Articulation/articulation_inference.py`、`Articulation/physics_estimation.py`、`DataGeneration/base_task.py`、`DataGeneration/pick_banana.py`、`utils/icp.py`	MIT；**实验性但最相关**；当前公开版组件不全	必选。你要复现的“论文工程骨架”就在这里，但要注意它当前是 Genesis 路线、相机对齐未开源	[18]
你的目标仿真器	`haosulab/SAPIEN`	articulation / physics / ray tracing docs；issue #282	仓库检索片段未直接显示统一 LICENSE，源码检索片段显示 Apache-2.0 头；持续有 2025–2026 issue 活跃	必选。承担 robot/object/seg/depth/physics	[19]

功能分类	推荐仓库	关键文件 / 脚本	License / 活跃度	选型意见	来源
SAPIEN 工程封装	`haosulab/ManiSkill`	`docs/.../custom_robots.md`、`mani_skill/agents/base_agent.py`	Apache-2.0；高活跃	强烈推荐。便于把 URDF、控制器、传感器、相机打包成可复用 agent	20
现有参考桥接	`qureshinomaan/SplatSim`	`scripts/launch_nodes.py`、`scripts/run_env_sim.py`、`scripts/articulated_robot_pipeline.py`、`configs/object_configs/objects.yaml`	已检索片段未明确显示 license；中等活跃	强烈推荐。不是 SAPIEN，但它提供了“GS 叠加在传统模拟器上”的最直接工程范式	21
3DGS 训练参考	`graphdeco-inria/gaussian-splatting`	`convert.py`、`train.py`	官方研究许可证；高星但许可证更严格	作为“训练参考”和结果对照非常好；不建议作为你项目的内嵌依赖首选	22
3DGS 项目内嵌渲染	`nerfstudio-project/gsplat`	Python API / LICENSE	Apache-2.0；高活跃	**你项目里首选的 GS 渲染库**。比官方实现更适合嵌入和分发	23
背景重建 / Nerf 工具链	`nerfstudio-project/nerfstudio`	`splatfacto`/method docs	官方活跃；`RoboSimGS` README 明确引用	推荐。若你想贴近论文 README 中的“Background Reconstruction”	24
相机与 SfM	`colmap/colmap`	COLMAP main repo	新 BSD；成熟、标准工具链	必选。负责真实视频到相机位姿/稀疏点云	25
自动分割 / 追踪	`IDEA-Research/Grounded-SAM-2`	repo main、相关 license files	多许可证文件；GroundingDINO 相关文件为 Apache-2.0；高活跃	推荐。做部件分割、mask 跟踪、视频自动化标注	26
几何处理	`isl-org/Open3D`	点云几何与可视化	MIT；成熟	必选。ICP、点云裁剪、mesh/point cloud I/O、可视化	27
几何分析	`mikedh/trimesh`	`LICENSE.md`、主仓库	MIT；成熟	必选。volume / inertia / mesh 清洗 / 质量分配	28
纹理随机化 / 合成数据	`DLR-RM/BlenderProc`	examples / material randomizer	GPL-3.0；高星	可选。若你需要更系统的材质随机化、镜头畸变、数据合成增强	29

功能分类	推荐仓库	关键文件 / 脚本	License / 活跃度	选型意见	来源
材料抽取	`astra-vision/MaterialPalette`	`LICENSE`、README	MIT；中高活跃	推荐。最适合“从真实图像抽取材质/PBR 线索”	[30]
PBR 资产生成	`Tencent-Hunyuan/Hunyuan3D-2.1`	`hy3dpaint`、`hy3dshape`、README	Tencent 社区许可；高星	可选。适合快速生成带 PBR 的资产，但要严格审查许可证	[31]
物理属性先验参考	`Maxwell-Zhao/PhysSplat`	README	检索片段未明确显示 license；实验性	推荐阅读。它与 RoboSimGS 共享 MLLM-guided physics 思路，可直接借鉴参数接口设计	[32]
负向可供性 / 细分先验	`Maxwell-Zhao/AffordDex`	README、`NAA` 目录	MIT；实验性	可选。适合做“不该抓哪里”的负样本标注或接触区域筛选	[33]
官方对标基座	`Genesis-Embodied-AI/Genesis`	README、`genesis/engine/simulator.py`	Apache-2.0；高活跃	推荐只做“参考基线”。你不嵌入它，但要对照它与 RoboSimGS 的接口差异	[34]

如果你要把这些仓库一次性拉到本地，下面这组命令最实用：

```
mkdir -p third_party && cd third_party

git clone https://github.com/Maxwell-Zhao/RoboSimGS.git
git clone https://github.com/haosulab/SAPIEN.git
git clone https://github.com/haosulab/ManiSkill.git
git clone https://github.com/qureshinomaan/SplatSim.git
git clone https://github.com/graphdeco-inria/gaussian-splatting.git
git clone https://github.com/nerfstudio-project/gsplat.git
git clone https://github.com/nerfstudio-project/nerfstudio.git
git clone https://github.com/colmap/colmap.git
git clone https://github.com/IDEA-Research/Grounded-SAM-2.git
git clone https://github.com/isl-org/Open3D.git
git clone https://github.com/mikedh/trimesh.git
git clone https://github.com/astra-vision/MaterialPalette.git
```

这组仓库已经覆盖了你要的四件套：**官方论文主线、SAPIEN 接入、背景 3DGS、物理/纹理/自动标注工具链**。[35]

端到端实施方案

先给出我建议你真正落地的端到端 pipeline。它不是对论文代码的逐文件搬运，而是对**已发布 GitHub 实现**做工程化重构后的最短路径。其核心思想是：**现实采集与资产生成阶段尽量贴近论文；仿真与数据生成阶段切到 SAPIEN；背景 3DGS 通过独立 renderer 合成。** [36]

```
flowchart LR
    A[真实场景采集<br/>手机视频/多视角照片] --> B[COLMAP<br/>相机位姿与稀疏点云]
    B --> C[背景 3DGS 训练<br/>gsplat / gaussian-splatting]
    A --> D[真实物体扫描<br/>mesh/GLB/OBJ]
    D --> E[部件分割<br/>Grounded-SAM-2 / 交互 SAM]
    E --> F[URDF 生成<br/>link joint limits damping friction]
    D --> G[物理属性估计<br/>caption+CLIP+LLM+material table]
    B --> H[真实/仿真坐标对齐<br/>Open3D ICP + 手动尺度]
    F --> I[SAPIEN 资产加载器<br/>URDF + visual mesh + collision]
    G --> I
    H --> I
    C --> J[GS 背景渲染器<br/>同步相机内外参]
    I --> K[SAPIEN 渲染<br/>RGB depth seg qpos action]
    J --> L[深度合成<br/>GS background + SAPIEN foreground]
    K --> L
    L --> M[自动数据集导出<br/>images actions qpos masks cameras]
    M --> N[zero-shot 训练/部署]
```

建议步骤

第一步：现实场景与视频采集。
如果你要贴近 `SplatSim` 与 `RoboSimGS` 的经验，必须让场景对 SfM 友好：背景最好有纹理、尽量使用漫反射光照、采集视频覆盖各个角度，并尽量避免在机器人包围盒内放置多余物体。`SplatSim` README 对采集条件写得很清楚；`RoboSimGS` README 则明确其背景先做 3DGS，再做 world-frame alignment。 [37]

第二步：生成背景 3DGS。
如果你追求最贴近论文 README，可走 `Nerfstudio/Feature Splatting`；如果你更关注可嵌入性，建议走 `COLMAP + gsplat` 或 `COLMAP + graphdeco train.py`。后者的最小命令链，`SplatSim` README 已经给出了一个可复制版本：先把视频转成图像，再通过 `convert.py` 跑结构光束与 COLMAP 数据整理，再用 `train.py` 训练 Gaussian Splats。 [38]

```
# 视频转帧
ffmpeg -i capture.MOV -qscale:v 1 data/bg_scene/input/frame_%04d.jpg

# 方案 A：官方 gaussian-splatting 训练参考
python third_party/gaussian-splatting/convert.py -s data/bg_scene
python third_party/gaussian-splatting/train.py -s data/bg_scene

# 训练输出通常包含 point_cloud/iteration_xxxxx/point_cloud.ply
```

如果你最终要把它嵌入 SAPIEN 项目，**训练可以用官方实现，嵌入渲染最好用 gsplat**。这个分工能兼顾“复现对照”和“工程落地”。 [39]

第三步：生成真实物体 mesh 与 articulation。
官方 README 把物体重建描述成手机扫描得到高质量 mesh；已发布 `articulation_inference.py` 则假定你有一个 `GLB` 输入，然后通过交互式部分分割与 `URDFGenerationPipeline` 产出 URDF。对你的 SAPIEN 项目，建议把这一段固化成如下目录约定：`assets/objects/<obj>/visual.glb`、`collision.obj`、`parts/*.glb`、`robot.urdf`。[40]

```
# 以 RoboSimGS 的现有命令行为模板
python third_party/RoboSimGS/Articulation/articulation_inference.py \
 assets/objects/box/visual.glb \
 --interactive \
 --use-sam-clip \
 --use-gpt-params \
 --robot-name box_lid \
 --output assets/objects/box/generated
```

这一步的输出不要直接交给训练代码，而是先过一遍**人工复核**：检查 link 名称、关节轴、limit、damping、friction，以及 visual/collision mesh 是否一致。因为当前公开版本仍然混有交互分割、关键词启发式和 GPT 参数建议。[41]

第四步：估计物理参数并转成 SAPIEN 可消费格式。
官方 `physics_estimation.py` 已经把输出 JSON 的 schema 写得很清楚。你可以直接沿用这个 schema，再加两项工程字段：`mass_total` 与 `inertia_source`。建议做法是让视觉管线只负责给出 `density / young / poisson / friction`，真正的质量和惯量则交给 mesh 计算完成。[42]

```
python third_party/RoboSimGS/Articulation/physics_estimation.py \
 assets/objects/box/rgb_ref.png \
 openai/clip-vit-base-patch32
```

在你自己的项目里，把它整理为：

```
{
 "density_kg_m3": 720.0,
 "youngs_modulus_mpa": 1400.0,
 "poisson_ratio": 0.32,
 "static_friction": 0.55,
 "dynamic_friction": 0.42,
 "restitution": 0.02,
 "mass_total_kg": 0.48,
 "inertia_source": "trimesh_tetra"
}
```

上面这个 JSON 结构是我建议的工程接口，不是论文仓库现成格式；它的好处是能直接喂给 SAPIEN 的物理材质与 collision shape 初始化。[43]

第五步：做世界坐标与相机对齐。
这一段是复现成败的关键。`RoboSimGS` 当前公开的是 world-frame ICP，而 camera pose alignment 明确“正在更新”；`SplatSim` 则给出了更细的 CloudCompare 手工流程。因此我建议你把“世界对齐”和“相机对

齐”彻底分开：先做点云/机器人几何的 4×4 对齐，再从 COLMAP 或 GS 训练输出中取相机外参，把它们变换到同一世界系。 [44]

```
python third_party/RoboSimGS/utils/icp.py \
 data/real_robot_segmented.ply \
 data/sim_robot_gt.ply \
 0.70
```

建议你将其结果保存为 `transforms/world_from_real.json`：

```
{
 "matrix_4x4": [[...], [...], [...], [...]],
 "source": "open3d_icp",
 "scale_manual": 0.70
}
```

第六步：把资产接入 SAPIEN。
SAPIEN 侧最小可运行示例如下。这个代码片段不是为了 1:1 复刻 Genesis，而是给你一个“稳定的 SAPIEN 资产入口”。下面的接口直接对应 SAPIEN 官方 articulation / physics / rendering 文档。 [45]

```
import json
from pathlib import Path
import sapien

root = Path("assets/objects/box/generated")

engine = sapien.Engine()
renderer = sapien.SapienRenderer()
engine.set_renderer(renderer)

scene = engine.create_scene()
scene.set_timestep(1 / 240.0)
scene.default_physical_material = scene.create_physical_material(
    static_friction=0.5,
    dynamic_friction=0.4,
    restitution=0.02,
)

# 加载 articulated object
loader = scene.create_urdf_loader()
obj = loader.load(str(root / "robot.urdf"))

# 可选：给可见 mesh 显式指定 PBR 贴图
texture = sapien.render.RenderTexture2D(str(root / "textures" / "albedo.png"))
material = sapien.render.RenderMaterial()
material.set_base_color_texture(texture)
material.base_color = [1, 1, 1, 1]
material.metallic = 0.1
```

```
material.roughness = 0.7

# 建议：如果 visual/collision 分离，则通过 actor_builder/builders 单独挂 visual mesh
# 并把 density/friction 等从 physics.json 映射到 scene physical material / collision shapes
```

如果你当前项目基于 ManiSkill，建议把上面的资产加载逻辑包装进自定义 `Agent` 或 object loader，而不是散落在 task 脚本里。ManiSkill 文档已经说明，导入 URDF/MJCF 后，controller、keyframe、sensors 这些仍要自己定义。[46]

第七步：3DGS 背景与 SAPIEN 前景做深度合成。
这是整份报告里最值得你做成单独模块的一步。做法很简单：SAPIEN 仍然输出 robot/object 的 `rgb/depth/segmentation`；GS 渲染器根据**相同内参/外参**输出背景 `rgb_bg/depth_bg`；最后按深度做 z-composite。`SplatSim` 已经证明“Gaussian Splatting 作为外层 renderer”可行；你的差别只是在底层从 PyBullet/Genesis 换成 SAPIEN。[47]

```
def composite(sim_rgb, sim_depth, sim_seg, gs_rgb, gs_depth, bg_entity_id=0, eps=1e-4):
    # sim_seg[..., 1] 在 SAPIEN issue 中被描述为 entity_id/link_id
    fg_mask = (sim_seg[..., 1] != bg_entity_id)
    depth_mask = (sim_depth < (gs_depth - eps))
    mask = fg_mask | depth_mask
    out = gs_rgb.copy()
    out[mask] = sim_rgb[mask]
    return out
```

这一步有两个关键前提。第一，**相机坐标必须统一**；第二，**不要依赖 SAPIEN 内部 entity_id 转 link_name 做语义标签**，而要你自己维护映射表。[48]

数据流与接口定义

下面这张表不是“论文原表”，而是我为你基于 SAPIEN 的项目建议的**稳定接口规范**。它尽量对齐了 `RoboSimGS` 已发布输出、SAPIEN/ManiSkill 的导入方式，以及 `COLMAP / gsplat / Open3D / trimesh` 的典型输入输出。[49]

阶段	输入	输出	坐标系 / 单位	建议文件格式	SAPIEN 接口落点
真实采集	视频 / 多视角图像	图像序列	相机原始坐标；像素	`jpg/png`	无
SfM / 相机恢复	图像序列	相机内外参、稀疏点云	世界系建议设为 COLMAP 世界系；长度米	`cameras.json`、`images.txt/bin`、`ply`	`scene.add_camera(...)` 参数来源

阶段	输入	输出	坐标系 / 单位	建议文件格式	SAPIEN 接口落点
背景 3DGS	图像 + 相机	GS 背景模型	与 SfM 世界系一致	`point_cloud.ply` / GS checkpoint	外部 `gsplat` renderer
物体 mesh	扫描 GLB/OBJ/mesh	清洗后的 visual mesh / collision mesh	米；右手系；z-up 或统一到项目约定	`visual.glb`、`collision.obj`	`add_visual_from_file` / loader
部件分割	完整 mesh / 图像	part meshes	继承物体局部坐标	`parts/*.glb`	URDF 生成前步骤
Articulation	parts + 关节参数	`robot.urdf`	关节角用弧度；位移用米	`urdf`	`scene.create_urdf_loader()`
物理估计	参考图像 + 材料先验	`physics.json`	`kg/m^3`、`MPa`、弧度、N・m	`json`	`create_physical_material`、collision density、自定义 mass/inertia
世界对齐	real/sim 点云	`T_world_from_real`	4×4 SE(3)；米	`json`	导入 pose / camera 外参转换
数据导出	SAPIEN render + GS render	RGB/Depth/Seg/Actions/Qpos	世界系统一	`png/npy/json/mp4`	训练与部署输入

下面这张表是**你迁移到 SAPIEN 必须动的改动清单**。它把 RoboSimGS/Genesis 的现状与 SAPIEN 对应点一一列出来。 [50]

改动点	RoboSimGS 当前做法	适配到 SAPIEN 的做法	必要性	备注
场景创建	`gs.Scene(...)`	`engine.create_scene()`	必须	这是最大断点
物体添加	`scene.add_entity(material=gs.materials.Rigid(), morph=gs.morphs.Mesh(...))`	`create_actor_builder()` / `URDF loader` / `ArticulationBuilder`	必须	visual 与 collision 建议分离
机器人控制	Genesis IK + `control_dofs_position`	SAPIEN joint drive / 你现有 controller	必须	若用 ManiSkill，更易封装

改动点	RoboSimGS 当前做法	适配到 SAPIEN 的做法	必要性	备注
world 对齐	Open3D ICP + 人工 scale	同样保留；输出 4×4 到 SAPIEN loader	必须	这部分可直接复用
camera pose alignment	官方未开源	采用 COLMAP / 手眼标定 / 统一外参文件	必须	要自己补完
GS 背景渲染	自定义 `Gaussians.render.Renderer`	`gsplat` 独立渲染 + 深度合成	强烈建议	不建议硬塞入 SAPIEN
光照增强	随机灯光颜色与强度	SAPIEN light + RenderMaterial / ray tracing 配置	推荐	数据多样性靠它
相机扰动	`reset_cam` 随机扰动	SAPIEN camera pose jitter	推荐	与论文公开 demo 一致
动力学参数	physics_estimation 输出 JSON，但 demo 更偏刚体	`physics.json` → physical material + density + mass/inertia	必须	建议你显式建接口
语义标签	依赖引擎内部对象	维护 `link_name -> stable_id` 映射表	必须	SAPIEN 3 当前存在 ID 映射缺口

验证、消融、性能与工程注意事项

消融与验证建议方面，至少要做四组。第一组是**视觉一致性**：固定相机位姿，对比真实图像、GS 背景渲染、SAPIEN 前景、最终合成图，计算 LPIPS/PSNR/SSIM 或至少做逐帧人工对比。第二组是**几何一致性**：对比真实点云与 SAPIEN 资产点云的 Chamfer/最近邻距离，确保 `icp.py` 输出不是“看起来差不多”而是真正对齐。第三组是**动力学一致性**：同一物体做简单 drop / slide / push 试验，调整 friction、density、Young's modulus，直到接触结果与真实视频方向一致。第四组是**策略鲁棒性**：对比“只用 SAPIEN 背景”“GS+SAPIEN 合成背景”“带材质扰动”和“不带材质扰动”的 zero-shot 成功率。前两组是必要验证，后两组是论文复现价值的体现。[51]

性能上，你需要把 bulk data generation 与 high-fidelity validation 分开。SAPIEN 文档说明其默认 rasterization 更适合数据密集任务，而 ray tracing 虽更真实，但代价更高；此外，2025 年一个 SAPIEN issue 中，用户在 ray tracing + 多光源配置下报告 `camera.take_picture()` 单次大约 0.136 秒。这对大规模数据生成会成为瓶颈。因此，我建议：**批量采集阶段用 rasterization，验证集或小批量高保真评估再切 ray tracing**。[52]

纹理工程上，尽量使用**显式 UV 贴图**，不要把关键资产依赖在 vertex color PLY/OBJ 上。因为 SAPIEN 公开 issue 里已有关于“顶点颜色/PLY/OBJ 颜色不显示”的用户反馈；另外，也有“多次替换 texture 后渲染不更新”与“文本ured box GPU memory 持续累积”的 issue。换言之，如果你要长期跑数据集生产，**纹理管理一定要做缓存、复用和生命周期管理**，并把“烘焙好的贴图文件夹”视为一等公民。[53]

并行化与内存方面，如果你打算同时跑 SAPIEN 与 GS 渲染器，最稳的做法不是“每个 environment 一个 GS renderer”，而是**单机场景、单 GS 模型、多相机帧批渲染**。这主要是因为 3DGS 本身显存开销就不小，而 `RoboSimGS` /Genesis 路线也展示了为了渲染和物理场景管理要在 GPU 上维护两套状态。若你还想引入软体/PBD/MPM 物体，建议放到第二阶段再做，因为 Genesis 自身在 2025–2026 的公开 issue 中仍有 soft materials 并行化和 PBD camera rendering 的问题，这说明“高保真 + 柔性体 + 大并行”的稳定工程难度本来就不低。[54]

风险与局限

最大的风险不是算法，而是**发布状态与接口断裂**。`RoboSimGS` 当前确实已经发布了 3D reconstruction、articulation inference、physics estimation、world coordinate alignment、simulated data generation 等模块，但 README 同时明确写了 **installation 仍在补充**、**camera pose alignment 尚未发布**。因此，任何“完全忠实、一键复现论文全部细节”的承诺都不可靠；更准确的说法是：**你可以复现论文的大部分工程思想与主链路，但必须自己补齐 camera alignment 与 SAPIEN adapter**。[55]

第二个风险来自**许可证**。如果你的项目只做学术复现，选择空间大很多；但如果你考虑产品化或可分发 SDK，许可证会直接决定你能不能用。官方 `gaussian-splatting` 的许可证明显比 Apache/MIT 更严格；`BlenderProc` 是 GPL-3.0；`Hunyuan3D-2.1` 也不是常规宽松许可证。相对而言，`gsplat`、`Open3D`、`trimesh`、`MaterialPalette`、`ManiSkill` 在工程上更友好。你应该在项目一开始就把“训练工具依赖”和“运行时依赖”分开，避免把限制性许可证带进最终 runtime。[56]

第三个风险是**SAPIEN 当前对 segmentation ID 到 link name 的运行时桥接并不完善**。公开 issue 表明，这一接口在 SAPIEN 3 中至少对一部分用户仍不清晰。对需要严肃做标签、接触分析、交互监督的数据工程来说，这不是小事。我建议你在第一版就把“稳定实例 ID 表”做成项目协议，而不要把数据语义绑死在引擎内部实现细节上。[48]

最后一个局限是**公开代码更偏向刚体抓取 demo，而不是完整通用物理对象库**。尽管论文方向覆盖了材料和物理属性估计，但当前发布的 `DataGeneration` 代码片段更明显地展示了刚体 mesh、plane、box、IK、相机抖动、灯光增强，这意味着你把论文路线嵌进 SAPIEN 时，最好先做出一个“刚体/准刚体的稳定版本”，再往 deformable/PBD/MPM 扩展。这样才能保证 zero-shot 部署链路先跑通。[57]

---

[1] [6] [7] [10] [18] [24] [35] [36] [40] [44] [49] [55] https://github.com/Maxwell-Zhao/RoboSimGS/tree/main
https://github.com/Maxwell-Zhao/RoboSimGS/tree/main

[2] [17] [21] [37] [38] [39] [47] https://github.com/qureshinomaan/SplatSim
https://github.com/qureshinomaan/SplatSim

[3] [51] https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/utils/icp.py
https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/utils/icp.py

[4] [12] [19] [45] https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/basic/create_articulations.rst
https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/basic/create_articulations.rst

[5] [14] [42] https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/Articulation/physics_estimation.py
https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/Articulation/physics_estimation.py

[8] [41] https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/Articulation/articulation_inference.py
https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/Articulation/articulation_inference.py

[9] [26] https://github.com/IDEA-Research/Grounded-SAM-2
https://github.com/IDEA-Research/Grounded-SAM-2

11 50 54 57 https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/DataGeneration/base_task.py
https://github.com/Maxwell-Zhao/RoboSimGS/blob/main/DataGeneration/base_task.py

13 34 https://github.com/Genesis-Embodied-AI/Genesis
https://github.com/Genesis-Embodied-AI/Genesis

15 28 https://github.com/mikedh/trimesh/blob/main/LICENSE.md
https://github.com/mikedh/trimesh/blob/main/LICENSE.md

16 30 https://github.com/astra-vision/MaterialPalette/blob/master/LICENSE
https://github.com/astra-vision/MaterialPalette/blob/master/LICENSE

20 46 https://github.com/haosulab/ManiSkill/blob/main/docs/source/user_guide/tutorials/custom_robots.md
https://github.com/haosulab/ManiSkill/blob/main/docs/source/user_guide/tutorials/custom_robots.md

22 https://github.com/graphdeco-inria/gaussian-splatting
https://github.com/graphdeco-inria/gaussian-splatting

23 https://github.com/nerfstudio-project/gsplat
https://github.com/nerfstudio-project/gsplat

25 https://github.com/colmap/colmap
https://github.com/colmap/colmap

27 https://github.com/isl-org/Open3D/blob/main/cpp/open3d/geometry/PointCloud.h
https://github.com/isl-org/Open3D/blob/main/cpp/open3d/geometry/PointCloud.h

29 https://github.com/DLR-RM/BlenderProc/blob/main/examples/README.md
https://github.com/DLR-RM/BlenderProc/blob/main/examples/README.md

31 https://github.com/tencent-hunyuan/hunyuan3d-2.1
https://github.com/tencent-hunyuan/hunyuan3d-2.1

32 https://github.com/Maxwell-Zhao/PhysSplat
https://github.com/Maxwell-Zhao/PhysSplat

33 https://github.com/Maxwell-Zhao/AffordDex
https://github.com/Maxwell-Zhao/AffordDex

43 https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/basic/physics.rst
https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/basic/physics.rst

48 https://github.com/haosulab/SAPIEN/issues/282
https://github.com/haosulab/SAPIEN/issues/282

52 https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/rendering/raytracing_renderer.rst
https://github.com/haosulab/sapien/blob/master/docs/source/tutorial/rendering/raytracing_renderer.rst

53 https://github.com/haosulab/SAPIEN/issues/208
https://github.com/haosulab/SAPIEN/issues/208

56 https://github.com/nerfstudio-project/gsplat/blob/main/LICENSE
https://github.com/nerfstudio-project/gsplat/blob/main/LICENSE